

CSA0404 OPERATING SYSTEM

LAB PROGRAM

NAME: KAMESH D

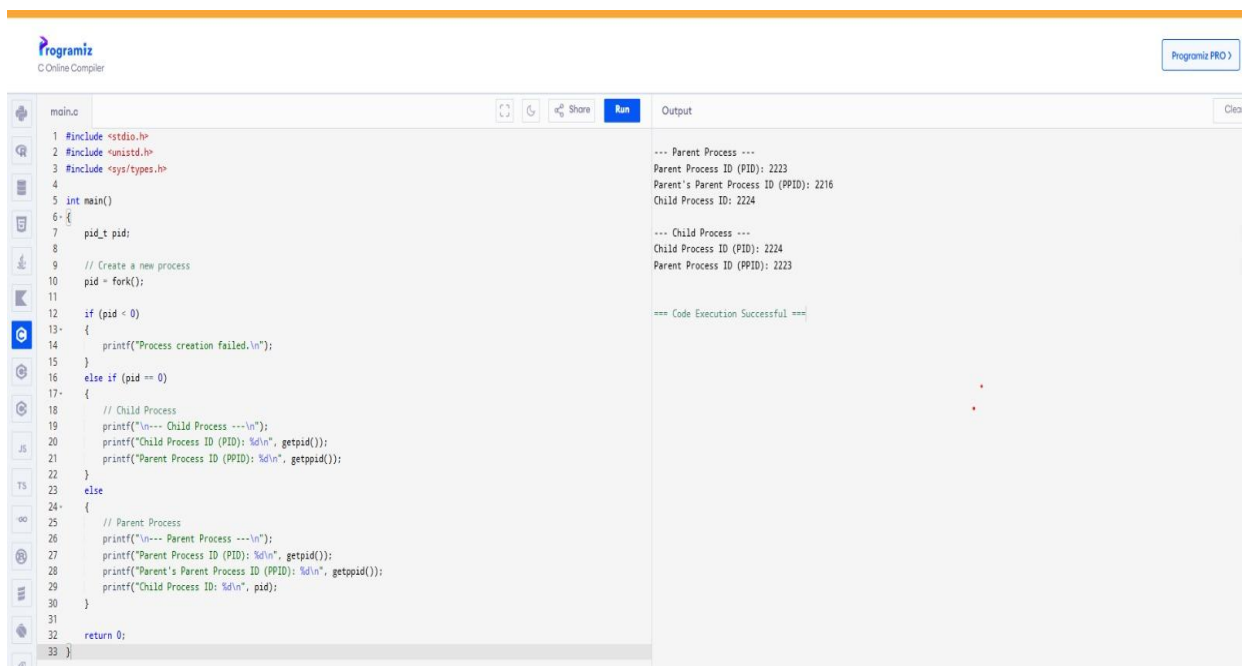
REG NO:192565003

PROGRAMS:

QUESTION 1

Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

CODING :



```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main()
6 {
7     pid_t pid;
8
9     // Create a new process
10    pid = fork();
11
12    if (pid < 0)
13    {
14        printf("Process creation failed.\n");
15    }
16    else if (pid == 0)
17    {
18        // Child Process
19        printf("\n--- Child Process ---\n");
20        printf("Child Process ID (PID): %d\n", getpid());
21        printf("Parent Process ID (PPID): %d\n", getppid());
22    }
23    else
24    {
25        // Parent Process
26        printf("\n--- Parent Process ---\n");
27        printf("Parent Process ID (PID): %d\n", getpid());
28        printf("Parent's Parent Process ID (PPID): %d\n", getppid());
29        printf("Child Process ID: %d\n", pid);
30    }
31
32    return 0;
33 }
```

Output

```
--- Parent Process ---
Parent Process ID (PID): 2223
Parent's Parent Process ID (PPID): 2216
Child Process ID: 2224

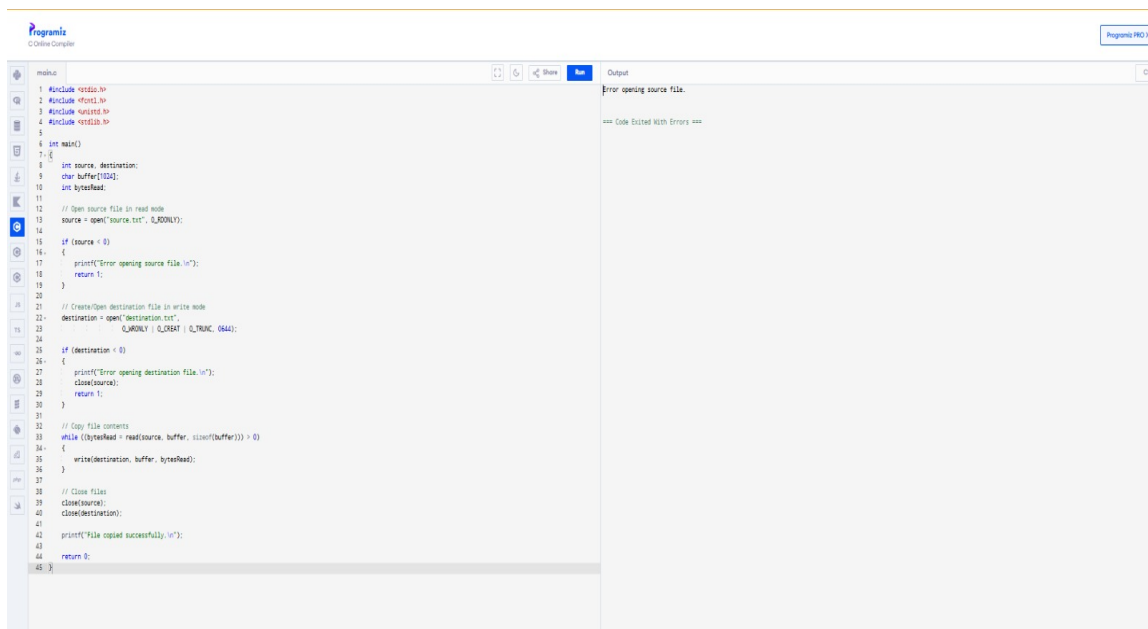
--- Child Process ---
Child Process ID (PID): 2224
Parent Process ID (PPID): 2223

=== Code Execution Successful ===
```

QUESTION 2:

Identify the system calls to copy the content of one file to another and illustrate the same using a C program

CODING:



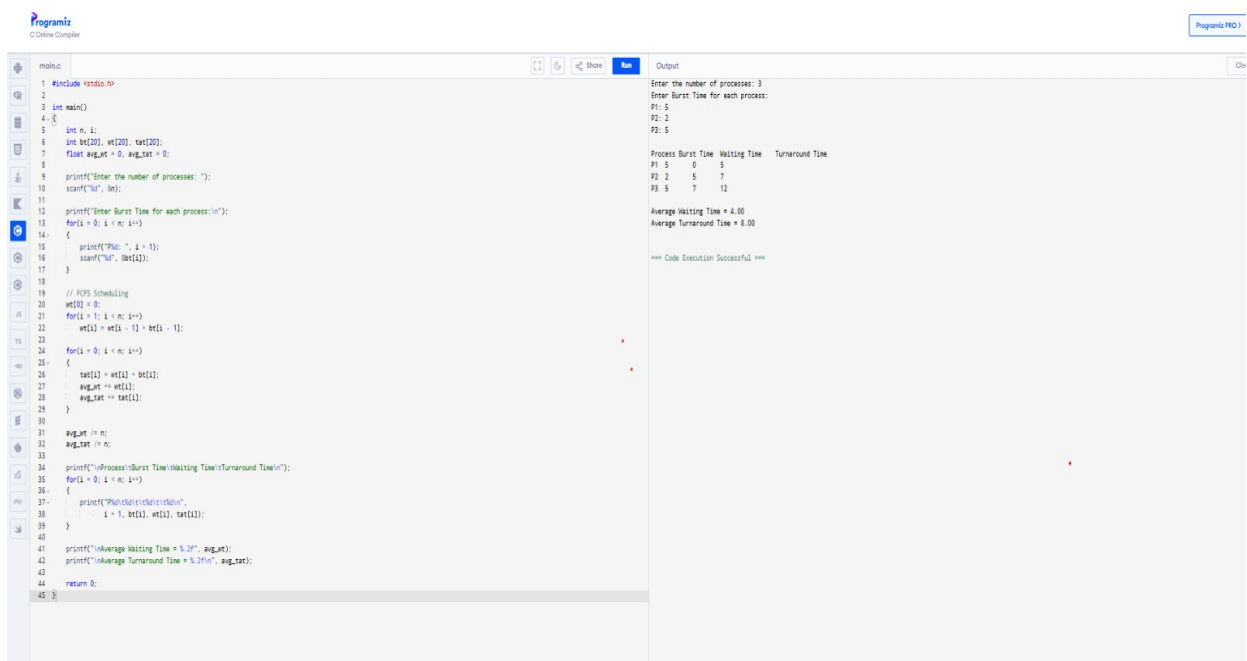
```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5
6 int main()
7 {
8     int source, destination;
9     char buffer[1024];
10    int bytesRead;
11
12    // Open source file in read mode
13    source = open("source.txt", O_RDONLY);
14
15    if (source < 0)
16    {
17        printf("Error opening source file.\n");
18        return 1;
19    }
20
21    // Create/Open destination file in write mode
22    destination = open("destination.txt",
23                      O_WRONLY | O_CREAT | O_TRUNC, 0644);
24
25    if (destination < 0)
26    {
27        printf("Error opening destination file.\n");
28        close(source);
29        return 1;
30    }
31
32    // Copy file contents
33    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
34    {
35        write(destination, buffer, bytesRead);
36    }
37
38    // Close files
39    close(source);
40    close(destination);
41
42    printf("File copied successfully.\n");
43    return 0;
44 }
```

Output: Error opening source file.
*** Code Executed With Errors ***

QUESTION 3:

Design a CPU scheduling program with C using First Come First Served.

CODING :



The screenshot shows a C program for CPU scheduling using the First Come First Served (FCFS) algorithm. The code is written in a C Online Compiler and includes comments for each step. The output shows the input data for three processes and the calculated metrics.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i;
6     int bt[20], wt[20], tat[20];
7     float avg_wt = 0, avg_tat = 0;
8
9     printf("Enter the number of processes: ");
10    scanf("%d", &n);
11
12    printf("Enter Burst Time for each process:\n");
13    for(i = 0; i < n; i++)
14    {
15        printf("P%d: ", i + 1);
16        scanf("%d", &bt[i]);
17    }
18
19    // FCFS Scheduling
20    wt[0] = 0;
21    for(i = 1; i < n; i++)
22        wt[i] = wt[i - 1] + bt[i - 1];
23
24    for(i = 0; i < n; i++)
25    {
26        tat[i] = wt[i] + bt[i];
27        avg_wt += wt[i];
28        avg_tat += tat[i];
29    }
30
31    avg_wt /= n;
32    avg_tat /= n;
33
34    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
35    for(i = 0; i < n; i++)
36    {
37        printf("P%d\t%d\t%d\t%d\n",
38               i + 1, bt[i], wt[i], tat[i]);
39    }
40
41    printf("\nAverage Waiting Time = %.2f", avg_wt);
42    printf("\nAverage Turnaround Time = %.2f", avg_tat);
43
44    return 0;
45 }
```

Output:

```
Enter the number of processes: 3
Enter Burst Time for each process:
P1: 5
P2: 2
P3: 5

Process Burst Time Waiting Time Turnaround Time
P1 5 0 5
P2 2 5 7
P3 5 7 12

Average Waiting Time = 4.00
Average Turnaround Time = 8.00

*** Code Execution Successful ***
```

QUESTION 4:

Construct a scheduling program with C that s

CODING:

The screenshot shows a C program for process scheduling using Shortest Burst Time (SJT) priority. The program takes the number of processes and their burst times as input, sorts them by burst time, and calculates the waiting and turnaround times for each process. The output shows the results for 3 processes with burst times 5, 2, and 5.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j;
6     int bt[20], wt[20], tat[20], p[20], temp;
7     float avg_wt = 0, avg_tat = 0;
8
9     printf("Enter the number of processes: ");
10    scanf("%d", &n);
11
12    printf("Enter Burst Time:\n");
13    for(i = 0; i < n; i++)
14    {
15        p[i] = i + 1;
16        printf("P%d: ", p[i]);
17        scanf("%d", &bt[i]);
18    }
19
20    // Sort processes by Burst Time (SJF)
21    for(i = 0; i < n - 1; i++)
22    {
23        for(j = i + 1; j < n; j++)
24        {
25            if(bt[i] > bt[j])
26            {
27                temp = bt[i];
28                bt[i] = bt[j];
29                bt[j] = temp;
30
31                temp = p[i];
32                p[i] = p[j];
33                p[j] = temp;
34            }
35        }
36    }
37
38    wt[0] = 0;
39    for(i = 1; i < n; i++)
40        wt[i] = wt[i - 1] + bt[i - 1];
41
42    for(i = 0; i < n; i++)
43    {
44        tat[i] = wt[i] + bt[i];
45        avg_wt += wt[i];
46        avg_tat += tat[i];
47    }
48
49    avg_wt /= n;
50    avg_tat /= n;
51}
```

Output:

```
Enter the number of processes: 3
Enter Burst Time for each process:
P1: 5
P2: 2
P3: 5

Process Burst Time Waiting Time Turnaround Time
P1 5 0 5
P2 2 5 7
P3 5 7 12

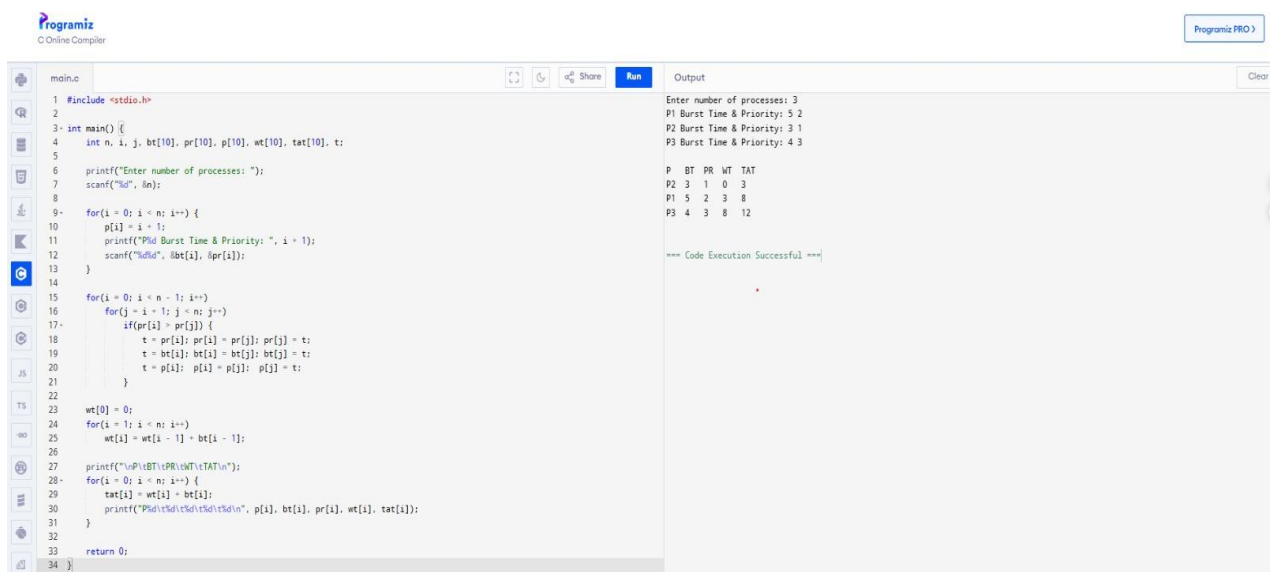
Average Waiting Time = 4.00
Average Turnaround Time = 8.00

=== Code Execution Successful ===
```

QUESTION 5:

Construct a scheduling program with C that selects the waiting process with the highest priority to execute next

CODING:



The screenshot shows a C program in the Programiz online compiler. The program implements a scheduling algorithm where processes are selected based on their priority. It uses arrays to store process details and a loop to find the process with the highest priority (lowest value) to execute next.

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, bt[10], pr[10], wt[10], tat[10], t;
5
6     printf("Enter number of processes: ");
7     scanf("%d", &n);
8
9     for(i = 0; i < n; i++) {
10         p[i] = i + 1;
11         printf("P%d Burst Time & Priority: ", i + 1);
12         scanf("%d%d", &bt[i], &pr[i]);
13     }
14
15     for(i = 0; i < n - 1; i++)
16         for(j = i + 1; j < n; j++)
17             if(pr[i] > pr[j]) {
18                 t = pr[i]; pr[i] = pr[j]; pr[j] = t;
19                 t = bt[i]; bt[i] = bt[j]; bt[j] = t;
20                 t = p[i]; p[i] = p[j]; p[j] = t;
21             }
22
23     wt[0] = 0;
24     for(i = 1; i < n; i++)
25         wt[i] = wt[i - 1] + bt[i - 1];
26
27     printf("\nP\tBT\tPR\tWT\tTAT\n");
28     for(i = 0; i < n; i++) {
29         tat[i] = wt[i] + bt[i];
30         printf("P%d\t%d\t%d\t%d\t%d\n", p[i], bt[i], pr[i], wt[i], tat[i]);
31     }
32
33     return 0;
34 }
```

The output of the program is as follows:

```
Enter number of processes: 3
P1 Burst Time & Priority: 5 2
P2 Burst Time & Priority: 3 1
P3 Burst Time & Priority: 4 3

P  BT  PR  WT  TAT
P2  3  1  0  3
P1  5  2  3  8
P3  4  3  8  12

=== Code Execution Successful ===
```